# FlareVM

*Windows-Based Malware Analysis and Forensic Arsenal*

**FlareVM** (Forensics, Logic Analysis, and Reverse Engineering) is a specialized Windows-based toolkit developed by the FLARE Team at Mandiant. It provides a comprehensive, pre-configured environment for reverse engineers, forensic investigators, and incident responders. This module explores the vast arsenal of tools available within FlareVM, ranging from **Static Analysis** utilities like **PEStudio** and **CFF Explorer** to **Dynamic Analysis** tools such as **Process Monitor (Procmon)** and **Process Explorer**.
We examine the methodology for identifying malicious indicators, including high **Entropy**, suspicious **API imports**, and **Command-and-Control (C2)** network activity. By mastering these tools, security professionals can effectively deconstruct malicious binaries and reconstruct the timeline of a digital breach.

This **documentation** and **learning** approach is strongly inspired by the structured, hands-on content provided by **TryHackMe**, which significantly contributed to practical understanding of "FlareVM Tool" concepts.

**DOCUMENTED BY**
Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

# 1. FlareVM Arsenal Categories

FlareVM centralizes hundreds of tools into logical categories to assist in various stages of a security investigation.

**Tool Categorization Table**

| Category | Primary Function | Key Tools |
|---|---|---|
| **Reverse Engineering** | Disassembling and debugging code. | **Ghidra**, **x64dbg**, **OllyDbg** |
| **Static/Dynamic Analysis** | Inspecting code vs. observing execution. | **PEStudio**, **Process Hacker**, **DIE** |
| **Forensics & IR** | Collecting and analyzing evidence. | **Volatility**, **FTK Imager**, **Rekall** |
| **Network Analysis** | Monitoring traffic and protocols. | **Wireshark**, **Nmap**, **Netcat** |
| **File Analysis** | Examining raw binary data. | **HxD**, **FileInsight**, **Hex Fiend** |
| **Sysinternals Suite** | Deep Windows system diagnostics. | **Procmon**, **Process Explorer**, **Autoruns** |

# 2. Core Investigation Tools

Initial investigations often rely on a "core" set of tools to identify high-level threats before moving to complex reverse engineering.

- **PEStudio:** The primary tool for **Static Analysis**. It allows analysts to view file properties (hashes, entropy, headers) without executing the file.
- **Process Monitor (Procmon):** Records real-time file system, registry, and process activity. Essential for seeing what a file does once opened.
- **Process Explorer (Procexp):** Shows parent-child relationships between processes. If a document spawns a shell, this tool identifies the hierarchy.
- **HxD:** A hex editor used to view the raw bytes of a file. It is used to identify **File Signatures** (e.g., 4D 5A for executables).
- **FLOSS:** The **FLARE Obfuscated String Solver**. It is used to automatically extract and de-obfuscate strings that standard "strings" tools cannot find.

WEIRDO

## 3. Key Indicators of Malicious Activity

When using static analysis tools, certain metrics frequently signal a threat:

- **Entropy:** A measure of randomness (0.0 to 8.0). High entropy (usually > 6.5) suggests the file is **Packed** or **Encrypted** to hide its code.
- **IAT (Import Address Table):** A list of functions the file requests from the OS. Blacklisted functions like set_UseShellExecute or CreateDecryptor are high-priority flags.
- **Metadata:** Suspicious descriptions or language markers (e.g., Russian metadata in a supposedly English system tool) indicate masquerading.

## 4. Practical Task Documentation (Task 3 & 4)

**I researched** the sample binaries provided in the Desktop\Sample folder and **found** several critical indicators of compromise. Below is how **I solved** the specific analysis challenges:

**Initial File Inspection (Task 3)**

- **Work Done: I practiced** using **PEStudio** on cryptominer.bin. **I founded** the **SHA256** hash and discovered the binary imported **51 functions**.
- **Hex Analysis: I researched** possible_medusa.txt using **CFF Explorer**. **I solved** the identification task by navigating to the **DOS Header** and **found** the e_magic value to be **5A4D** (indicating it is actually an executable despite the .txt extension). **I founded** the MD5 hash of this file to be 711E8D1488176840F27993099955353C.

**Malicious Binary Analysis (Task 4)**

- **Target: windows.exe**
  - **Work Done: I practiced** static analysis using **PEStudio**. **I founded** the **entropy value** to be **6.012**.
  - **Term Explanation: I researched** the **requestedExecutionLevel** in the manifest section and **found** it set to asInvoker.
  - **Function Identification: I solved** the API search and **founded** that set_UseShellExecute was present, allowing the malware to spawn other processes. **I also found** the RijndaelManaged API, which indicates the use of **AES Cryptography**.
- **Target: cobaltstrike.exe**
  - **Work Done: I practiced** dynamic analysis by executing the file and monitoring it with **Process Explorer**. **I founded** that the **parent process** of the malicious file was explorer.exe.
  - **Network Triage: I researched** the network activity and **solved** the C2 identification task. **I founded** that the process was connecting to the **C2 IP Address** 47.120.46.210 on **Port 443**.
  - **Verification: I practiced** using **Procmon** with a filter (Process Name contains cobalt) to confirm the outbound connection details.

WEIRDO

# Conclusion and Cheatsheet

**Conclusion**

FlareVM is a powerhouse for Windows-based defensive operations, providing the necessary tools to pivot from simple file identification to deep-dive behavior analysis. By using **PEStudio** for initial indicators and **Sysinternals** for execution monitoring, analysts can identify packed files and C2 communication patterns. Understanding how to interpret **Entropy**, **Import Address Tables**, and **Hex Headers** allows for rapid triage and effective incident response.

**Fast Revision Cheatsheet**

- **4D 5A (MZ):** The standard hex signature for a Windows executable.
- **Entropy:** randomness score; high values suggest packing/encryption.
- **PEStudio:** Best for safe, static inspection of binary properties.
- **Procmon:** Best for real-time monitoring of file and registry changes.
- **Procexp:** Best for viewing process trees and parent-child links.
- **FLOSS:** Use this when standard string extraction fails.
- **Import Address Table (IAT):** List of external functions used by a program.
- **requestedExecutionLevel:** Defines the privilege level requested from Windows.

WEIR